Vectorized SVE2 Optimization of the Post-Quantum Signature ML-DSA on ARMv9-A Architecture

Hanyu Wei, Wenqian Li, Shiyu Shen *Member, IEEE*, Hao Yang, Wenbo Guo, Yunlei Zhao

***Abstract*—Post-quantum cryptography (PQC) is essential to securing data in the quantum computing era, and standardization efforts led by NIST have driven extensive research on practical and efficient implementations. With the emerging deployment of ARMv9-A processors in mobile and edge devices, optimizing PQC algorithms for this architecture is becoming increasingly important. Among the NIST-selected digital signature schemes, ML-DSA stands out due to its strong security and efficiency, making it suitable for general purposes. In this work, we present a highly optimized implementation of ML-DSA for the ARMv9-A architecture, leveraging the SVE2 vector instruction set. We propose a vector-friendly sparse polynomial multiplication scheme and introduce an early-check mechanism that significantly reduces redundant computation in the signature validity check. We also design a tailored conditional instruction pipeline to further enhance efficiency. Our implementation achieves a 70.7% performance improvement in signature generation compared to the baseline implementation, establishing the first highly vectorized ML-DSA implementation on ARMv9-A by SVE2 extension. These results demonstrate the practicality of deploying high-performance post-quantum signatures on next-generation mobile and edge platforms.**

***Index Terms*—Post-quantum cryptography, polynomial multiplication, SVE2, ARMv9-A.**

I. Introduction

DIGITAL signatures are essential for secure authentication in online communications, enabling services such as website verification and software integrity checks. However, the emergence of large-scale quantum computers poses a critical threat to conventional public-key cryptosystems, as they are vulnerable to quantum attacks enabled by Shor's algorithm [1] and Grover's algorithm [2]. In response, post-quantum cryptography (PQC) has emerged as a promising solution, designed to resist attacks from quantum adversaries. Among various PQC approaches, lattice-based cryptography has attracted considerable attention due to its strong theoretical foundations and implementation efficiency. Dilithium [3], a lattice-based digital signature algorithm, exemplifies these advantages. It has been selected as the standardized digital signature algorithm by NIST in its PQC project, under the name FIPS 204 (ML-DSA) [4]. ML-DSA is based on the Fiat-Shamir with Aborts paradigm [5], [6], and its security is proven in the SUF-CMA (Strong Unforgeability under Chosen Message Attacks) model, based on the hardness of two fundamental lattice problems: module learning with errors (MLWE) [7] and module short integer solution (MSIS) [8]. Combining efficiency, simplicity, and strong security guarantees, ML-DSA is well-suited for deployment across a broad spectrum of platforms, from high-performance servers to embedded systems.

During the NIST PQC standardization process, many optimized implementations were proposed across various hardware platforms. Notably, significant efforts have targeted field programmable gate arrays (FPGA) [9]–[13], x86 CPUs with AVX2/AVX512 support [3], [14], [15], high-throughput GPUs [16]–[19], and resource-constrained ARM Cortex-M4 microcontrollers [20]–[24], which are also recommended as official reference platforms by NIST. Beyond these, researchers have explored implementations on processors such as ARM Cortex-A CPUs [25]–[27], reflecting the growing need to support PQC across diverse and evolving hardware ecosystems. These developments underscore the importance of cross-platform optimization to enable practical, real-world deployment of PQC.

Among ARM A-profile CPUs [28], the ARMv9-A architecture has emerged as a compelling platform for PQC applications, particularly in both commercial and high-performance computing domains. As the successor to ARMv8-A, ARMv9-A maintains support for fixed-length NEON (Advanced SIMD) instructions [29], while introducing the scalable vector extension version 2 (SVE2) [30]. This instruction set enables vector-length-agnostic programming and high-throughput vector operations, making it especially suited for exploiting the inherent parallelism of cryptographic workloads. Commercial adoption of ARMv9-A is accelerating. For example, Apple M4 series SoC [31] is the first publicly available processor to support SVE2. MediaTek Dimensity 7200, featured in the Vivo V27 smartphone [32], and Samsung Exynos 1580 [33] also incorporate ARMv9-A processors. Furthermore, Arm's own edge AI platforms [34] highlight the architecture's applicability to secure, power-efficient, and intelligent computing at the edge. This work is motivated by the observation that SVE2's vector-length-agnostic design and its higher integer-operation throughput can be exploited to accelerate polynomial arithmetic, which constitutes the primary performance bottleneck in ML-DSA. Given its increasing commercial adoption and advanced vector-processing capabilities, optimizing post-quantum cryptographic algorithms for ARMv9-A is both timely and essential. Such optimizations bridge the gap between cryptographic research and real-world deployment, ensuring that future secure systems can fully exploit the performance potential of modern ARM-based processors.

Corresponding author: Yunlei Zhao.

Hanyu Wei, Wenqian Li, Wenbo Guo and Yunlei Zhao are with School of Computer Science, Fudan University, Shanghai 200438, China (e-mail: hywei24@m.fudan.edu.cn; liwq24@m.fudan.edu.cn; guowenbo@fudan.edu.cn; ylzhao@fudan.edu.cn).

Shiyu Shen and Hao Yang are with Department of Electrical Engineering, City University of Hong Kong, Hong Kong, China (e-mail: crypto@sher1e.dev; crypto@d4rk.dev).

A. Related Work and Motivation

As one of the digital signature schemes standardized by NIST, ML-DSA has received extensive attention in both theoretical analysis and practical implementation. Numerous efforts have been made to optimize its performance across diverse hardware platforms [3], [14], [15], [20]–[22], [25], [26], leveraging vectorized instruction sets such as NEON, AVX2, and AVX512. We focus on related implementations targeting ARM-based architectures, with an emphasis on polynomial multiplication, one of the most computationally intensive components of ML-DSA. Several works [20]–[22] have explored the acceleration of Number Theoretic Transform (NTT)-based polynomial multiplication, modular reduction/multiplication on resource-constrained ARM M-profile processors, investigating algorithm-level and instruction-level optimizations to balance speed and memory consumption in Dilithium implementations on Cortex-M architectures.

On the A-profile side, Becker et al. [35] proposed a fast and widely adopted Keccak permutation implementation optimized for ARMv8-A CPUs using NEON. Our implementation inherits this Keccak module to ensure consistency and efficiency in the hashing components of Dilithium. Becker et al. [25] introduced a more efficient Barrett multiplication method tailored for NEON, which significantly accelerates NTT computation and improves the performance of polynomial multiplication. Zheng et al. [26] presented a parallel small polynomial multiplication technique, achieving noticeable speedups for Dilithium signature generation on ARMv8-A processors. Their method showcases the potential of parallelism in small-coefficient polynomial operations in Dilithium. Although NEON extension provides effective fixed-width SIMD acceleration on ARMv8-A, its fixed-width vector model and limited scalability constrain its ability to fully exploit data-level parallelism for polynomial arithmetic; by contrast, SVE2 extension enables more portable vectorization and higher throughput across diverse core implementations.

These works collectively demonstrate the importance of vectorized, architecture-specific optimizations for deploying PQC algorithms such as ML-DSA on ARM platforms. However, to the best of our knowledge, no prior work has investigated an implementation leveraging the SVE2 instruction set introduced in the ARMv9-A architecture. The ARMv9-A architecture is increasingly adopted in commercial processors, including Apple M4, Samsung Exynos 1580, and MediaTek Dimensity 7200. As these processors are used in performance-critical and security-sensitive scenarios, ensuring the efficient and secure deployment of PQC algorithms on this platform has become an urgent and practical challenge. Our work fills this gap by presenting the first SVE2-optimized ML-DSA implementation targeting modern ARMv9-A processors, aiming to improve the performance of PQC algorithms on emerging ARM platforms.

B. Contributions

We analyze the performance bottlenecks of ML-DSA and identify polynomial multiplication as one of the most critical components. To accelerate this key module, we discuss several optimization techniques. NTT methods reduce the complexity of polynomial multiplication from $\mathcal{O}(n^2)$ to $\mathcal{O}(n \log n)$. For the sparse polynomial and small-coefficient multiplication in ML-DSA, we introduce the parallel sparse polynomial multiplication (PSPM) method [16], [26], whose complexity is $\mathcal{O}(\tau n)$, where τ denotes the number of non-zero terms in the sparse polynomial. Implementing these methods on ARMv9-A introduces new challenges, and, to the best of our knowledge, there is currently no SVE2-based implementation; we present the first SVE2-optimized implementations of these polynomial multiplication methods.

We analyze the adaptation between scalable vector registers and polynomial lengths, and design inter-register and intra-register butterfly structures for NTT, while also vectorizing modular arithmetic to fit the computation model. For the PSPM method, we propose a chunk-based accumulation approach that leverages vectorization to significantly improve computational efficiency. Additionally, we adapt early rejection techniques [16] for PSPM to the ARMv9-A by storing results in predicate registers and combining them with bitwise operations to reduce branching. After optimizing polynomial multiplication, we select the most efficient method for signature generation. Our evaluation shows that PSPM outperforms NTT for sparse polynomial multiplication under SVE2 vectorization. Leveraging PSPM not only accelerates computation but also enables parallel execution of validity checks, thus improving pipeline efficiency.

This work presents the first and highly vectorized implementation of ML-DSA on the ARMv9-A architecture using the SVE2 instruction set. Comprehensive evaluation on an ARMv9-A processor (e.g., Apple M4 Pro) demonstrates a 70.7% improvement in signature generation compared to the reference implementation [4]. To the best of our knowledge, this SVE2-based implementation currently achieves the best known performance for ML-DSA on the ARMv9-A platform.

In summary, our main contributions are as follows:

- We design a vector-friendly scheme for both the NTT-based multiplication and PSPM.
- We introduce a predicate-based adaptation for early rejection, further improving signature generation efficiency.
- We present the first and highly optimized implementation of ML-DSA on the ARMv9-A architecture by SVE2.

II. Preliminary Knowledge

A. Notations

We denote by $\mathbb{Z}$ the ring of integers, and by $\mathbb{Z}_q = \mathbb{Z}/q\mathbb{Z}$ the ring of integers modulo q. Let $R = \mathbb{Z}[X]/(X^n + 1)$ and $R_q = \mathbb{Z}_q[X]/(X^n + 1)$ be the polynomial rings. Bold italic lowercase letters (e.g., $\boldsymbol{c}$) denote elements of R or R_q, i.e., polynomials $\boldsymbol{c} = \sum_{i=0}^{n-1} c_i X^i$ with coefficients $c_i \in \mathbb{Z}$ or $\mathbb{Z}_q$. Bold upright lowercase letters like $\mathbf{t}$ denote vectors whose entries are polynomials in R or R_q. Bold upright uppercase letters, such as $\mathbf{A}$, denote matrices whose entries are also polynomials in R or R_q. Notations with a hat symbol (e.g., $\hat{\boldsymbol{c}}$, $\hat{\mathbf{t}}$, $\hat{\mathbf{A}}$) refer to the corresponding polynomial, vector, or matrix after transformation into the NTT domain. The element-wise multiplication in $\mathbb{Z}_q$ is denoted by $\circ$.

We denote centered reduction modulo α by $\mathrm{mod}^{\pm}\alpha$. For an even positive integer α, the centered reduction of $r \in \mathbb{Z}$ is defined as the unique integer $r' = r \bmod^{\pm} \alpha$ satisfying $-\frac{\alpha}{2} < r' \leq \frac{\alpha}{2}$. If α is an odd positive integer, the result is instead in the range $-\frac{\alpha-1}{2} \leq r' \leq \frac{\alpha-1}{2}$. The ℓ_∞ norm of an element w is denoted by $\|w\|_\infty$. For $w \in \mathbb{Z}_q$, we define $\|w\|_\infty = |w \bmod^{\pm} q|$. For a polynomial $\boldsymbol{w} = \sum_{i=0}^{n-1} w_i X^i \in R$, we define the norm as $\|\boldsymbol{w}\|_\infty = \max_{0 \leq i < n} \|w_i\|_\infty$. Similarly, for a vector $\mathbf{w} = (\boldsymbol{w}_1, \ldots, \boldsymbol{w}_k) \in R^k$, we define $\|\mathbf{w}\|_\infty = \max_{0 \leq i < k} \|\boldsymbol{w}_i\|_\infty$. We define the set S_η as the set of all polynomials $\boldsymbol{w} \in R$ such that $\|\boldsymbol{w}\|_\infty \leq \eta$. We also define $\tilde{S}_\eta$ to be the set $\tilde{S}_\eta = \{\boldsymbol{w} \bmod^{\pm} 2\eta : \boldsymbol{w} \in R\}$. The sets S_η and $\tilde{S}_\eta$ are similar, except that S_η does not include any polynomial whose coefficients are equal to $-\eta$.

The notation tr denotes a bit string consisting of elements in $\{0,1\}$, and $\mathbb{B}^l$ denotes a byte string of length l. The expression "# of 1's in $\mathbf{h}$" denotes the number of entries equal to 1 in $\mathbf{h}$.

B. Digital Signature of ML-DSA

ML-DSA [4] adopts the Fiat-Shamir with Aborts construction [5], [6]. It shares structural similarities with earlier lattice-based schemes [36], [37]. The scheme operates over the polynomial ring $R_q = \mathbb{Z}_q[X]/(X^n + 1)$, with $n = 256$ and modulus $q = 2^{23} - 2^{13} + 1 = 8380417$. Polynomial multiplication constitutes the primary computational bottleneck in ML-DSA, especially during signature generation and verification. This operation is accelerated using the Number-Theoretic Transform (NTT) for efficient convolution in R_q. ML-DSA digital signature [4] consists of three core procedures: KeyGen, Sign, and Verify, each outlined in Algorithm 1, 2 and 3. The three parameter sets, ML-DSA-44, ML-DSA-65, and ML-DSA-87, correspond to NIST security levels 2, 3, and 5, respectively, as summarized in Table I. We omit the definitions of auxiliary functions such as Power2Round, HighBits, MakeHint, and UseHint; for detailed descriptions, we refer the reader to the algorithm specification [4].

Algorithm 1 ML-DSA.KeyGen

Ensure: Public key $pk = (\rho, \mathbf{t}_1)$ and private key $sk = (\rho, \mathsf{K}, \mathsf{tr}, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_0)$.

$\xi \leftarrow \mathbb{B}^{32}$ ▷ choose random seed
$(\rho, \rho', \mathsf{K}) \in \mathbb{B}^{32} \times \mathbb{B}^{32} \times \mathbb{B}^{32} := \mathsf{H}(\xi)$
$\hat{\mathbf{A}} \in R_q^{k \times \ell} := \mathsf{ExpandA}(\rho)$ ▷ $\mathbf{A}$ is generated and stored in NTT representation as $\hat{\mathbf{A}}$
$(\mathbf{s_1}, \mathbf{s_2}) \in S_\eta^\ell \times S_\eta^k := \mathsf{ExpandS}(\rho')$
$\mathbf{t} := \mathsf{NTT}^{-1}(\hat{\mathbf{A}} \circ \mathsf{NTT}(\mathbf{s_1})) + \mathbf{s_2}$
$(\mathbf{t_1}, \mathbf{t_0}) := \mathsf{Power2Round}(\mathbf{t})$
$\mathsf{tr} \in \mathbb{B}^{48} := \mathsf{CRH}(\rho \| \mathbf{t}_1)$
return $(pk = (\rho, \mathbf{t}_1), sk = (\rho, \mathsf{K}, \mathsf{tr}, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_0))$

C. Scalable Vector Extension 2 (SVE2)

SVE2 is a vector instruction set extension to the AArch64 architecture, introduced as a superset of both the original SVE and the NEON SIMD architecture. It enhances data-level parallelism capabilities and is a key feature of the ARMv9-A architecture. SVE2 enables efficient vectorization across a broad range of application domains, including high-performance computing (HPC), computer vision, signal processing, and general-purpose software.

SVE2 introduces a scalable register file and control registers that extend the AArch64 register set. It includes 32 scalable vector registers `Z0-Z31` used for vector operands. Their bit-width is implementation-dependent, ranging from 128 to 2048 bits. It has 16 predicate registers `P0-P15` used for per-lane predication to control which elements participate in an operation. One of the key features of SVE2 is predication, which allows fine-grained control over which lanes of a vector operation are active. Each instruction can be governed by

Algorithm 2 ML-DSA.Sign

Require: Private key $sk = (\rho, \mathsf{K}, \mathsf{tr}, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_0)$, message $\mathsf{M} \in \{0,1\}^*$.
Ensure: Signature $\sigma = (\mathbf{z}, \mathbf{h}, \tilde{\mathsf{c}})$.

$\hat{\mathbf{A}} \in R_q^{k \times \ell} \leftarrow \mathsf{ExpandA}(\rho)$
$\mu \in \mathbb{B}^{48} := \mathsf{CRH}(\mathsf{tr} \| \mathsf{M})$
$\kappa := 0, (\mathbf{z}, \mathbf{h}) := \perp$
$\rho'' \in \mathbb{B}^{48} := \mathsf{CRH}(\mathsf{K} \| \mu)$
while $(\mathbf{z}, \mathbf{h}) = \perp$ **do**
  $\mathbf{y} \in \mathcal{R}_q^\ell := \mathsf{ExpandMask}(\rho'', \kappa)$
  $\mathbf{w} := \mathsf{NTT}^{-1}(\hat{\mathbf{A}} \circ \mathsf{NTT}(\mathbf{y}))$
  $\mathbf{w_1} := \mathsf{HighBits}_q(\mathbf{w}, 2\gamma_2)$
  $\tilde{\mathsf{c}} \in \mathbb{B}^{32} := \mathsf{H}(\mu \| \mathbf{w_1})$
  $\boldsymbol{c} \in B_\tau := \mathsf{SampleInBall}(\tilde{\mathsf{c}})$
  $\hat{\boldsymbol{c}} := \mathsf{NTT}(\boldsymbol{c})$
  $\langle \boldsymbol{c}\mathbf{s}_1 \rangle := \mathsf{NTT}^{-1}(\hat{\boldsymbol{c}} \circ \mathsf{NTT}(\mathbf{s}_1))$
  $\langle \boldsymbol{c}\mathbf{s}_2 \rangle := \mathsf{NTT}^{-1}(\hat{\boldsymbol{c}} \circ \mathsf{NTT}(\mathbf{s}_2))$
  $\mathbf{z} \leftarrow \mathbf{y} + \langle \boldsymbol{c}\mathbf{s_1} \rangle$
  $\mathbf{r}_0 := \mathsf{LowBits}_q(\mathbf{w} - \langle \boldsymbol{c}\mathbf{s}_2 \rangle, 2\gamma_2)$
  if $\|\mathbf{z}\|_\infty \geq \gamma_1 - \beta$ or $\|\mathbf{r}_0\|_\infty \geq \gamma_2 - \beta$ **then**
    $(\mathbf{z}, \mathbf{h}) := \perp$
  else
    $\langle \boldsymbol{c}\mathbf{t}_0 \rangle := \mathsf{NTT}^{-1}(\hat{\boldsymbol{c}} \circ \mathsf{NTT}(\mathbf{t}_0))$
    $\mathbf{h} := \mathsf{MakeHint}_q(-\langle \boldsymbol{c}\mathbf{t}_0 \rangle, \mathbf{w} - \langle \boldsymbol{c}\mathbf{s}_2 \rangle + \langle \boldsymbol{c}\mathbf{t}_0 \rangle, 2\gamma_2)$
    if $\|\langle \boldsymbol{c}\mathbf{t}_0 \rangle\|_\infty \geq \gamma_2$ or # of 1's in $\mathbf{h}$ is $> \omega$ **then**
      $(\mathbf{z}, \mathbf{h}) := \perp$
  $\kappa := \kappa + \ell$
return $\sigma = (\mathbf{z}, \mathbf{h}, \tilde{\mathsf{c}})$

Algorithm 3 ML-DSA.Verify

Require: Public key $pk = (\rho, \mathbf{t}_1)$, message $\mathsf{M} \in \{0,1\}^*$, Signature $\sigma = (\mathbf{z}, \mathbf{h}, \tilde{\mathsf{c}})$.
Ensure: valid or invalid.

$\hat{\mathbf{A}} \in R_q^{k \times \ell} := \mathsf{ExpandA}(\rho)$
$\mu \in \mathbb{B}^{48} := \mathsf{CRH}(\mathsf{CRH}(\rho \| \mathbf{t}_1) \| \mathsf{M})$
$\boldsymbol{c} := \mathsf{SampleInBall}(\tilde{\mathsf{c}})$
$\mathbf{w}'_{\mathsf{Approx}} := \mathsf{NTT}^{-1}(\hat{\mathbf{A}} \circ \mathsf{NTT}(\mathbf{z}) - \mathsf{NTT}(\boldsymbol{c}) \circ \mathsf{NTT}(\mathbf{t}_1 \cdot 2^d))$
▷ $\mathbf{w}'_{\mathsf{Approx}} = \mathbf{A}\mathbf{z} - \boldsymbol{c}\mathbf{t}_1 \cdot 2^d$
$\mathbf{w}'_1 := \mathsf{UseHint}_q(\mathbf{h}, \mathbf{w}'_{\mathsf{Approx}}, 2\gamma_2)$
$\tilde{\mathsf{c}}' \leftarrow \mathsf{H}(\mu \| \mathbf{w}'_1)$
return $[\|\mathbf{z}\|_\infty < \gamma_1 - \beta]$ and [# of 1's in $\mathbf{h}$ is $\leq \omega$] and $[\tilde{\mathsf{c}} = \tilde{\mathsf{c}}']$

TABLE I
PARAMETER SETS OF ML-DSA.

Parameters	ML-DSA-44	ML-DSA-65	ML-DSA-87
q (modulus)	8380417	8380417	8380417
d (# of dropped bits from $\mathbf{t}$)	13	13	13
τ (# of ±1's in $\boldsymbol{c}$)	39	49	60
γ_1 (coefficient range of $\mathbf{y}$)	2^{17}	2^{19}	2^{19}
γ_2 (low-order rounding range)	$\frac{q-1}{88}$	$\frac{q-1}{32}$	$\frac{q-1}{32}$
(k,ℓ) (Dimensions of $\mathbf{A}$)	(4,4)	(6,5)	(8,7)
η (private key range)	2	4	2
β ($\tau\cdot\eta$)	78	196	120
ω (Max. # of 1's in the hint $\mathbf{h}$)	80	55	75
Repetitions	4.25	5.1	3.85

a predicate register (e.g., `P0`), which determines lane-level activity. For example, `ADD Z0.D, P0/M, Z1.D, Z2.D`, the addition is applied only to the lanes of `Z1` and `Z2` that are active according to `P0`. The modifier `M` indicates that inactive lanes in the destination `Z0` retain their original values, whereas using `Z` would zero those lanes.

In this work, we target the Apple M4 Pro SoC [31], the first publicly available ARMv9-A processor that supports SVE2. Using the `rdvl` instruction, we confirm that the default SVE2 vector length on this platform is 512 bits. This vector width aligns well with the data-level parallelism inherent in the ML-DSA algorithm, making it suitable for efficient implementation. On the Apple M4, SVE2 instructions are available only in *streaming SVE mode*, which must be explicitly activated using the `smstart` and `smstop` instructions. In this mode, NEON instructions are disabled. Although the M4 supports both NEON and SVE2, they cannot be executed concurrently within the same mode.

III. DESIGN FOR POLYNOMIAL MULTIPLICATION

A. Polynomial multiplication in ML-DSA

As discussed in Section II-B, we summarize the polynomial multiplications involved in ML-DSA along with their coefficient ranges or distributions in Table II. Matrix-vector multiplications such as $\mathbf{As}_1$, $\mathbf{Ay}$, and $\mathbf{Az}$ are performed in the NTT domain since the matrix $\mathbf{A}$ is sampled directly in this domain. To maintain consistency with the provided test vectors, we also compute these matrix-vector polynomial multiplications entirely in the NTT domain.

A particularly noteworthy component is the polynomial $\boldsymbol{c}\in B_\tau$, which is repeatedly used during signature generation. This polynomial is **sparse**, containing exactly τ non-zero coefficients, each of which is either $+1$ or -1, with all other coefficients being zero. The other multiplicands, specifically the polynomial vectors $\mathbf{s}_1$ and $\mathbf{s}_2$, are sampled from the distribution S_η, where each coefficient lies in the range $[-\eta,\eta]$ with η equal to 2 or 4. We refer to these as **small-coefficient** polynomials. In contrast, polynomials in $\mathbf{t}_0$ require a d-bit representation. The value range of $\mathbf{t}_0$ is $(-2^{\lceil\log_2 q\rceil-d}, 2^{\lceil\log_2 q\rceil-d}]$, as determined by the Power2Round function [3] and the parameter sets. This wider range introduces additional implementation challenges that require special handling. Below, we present customized optimization strategies tailored to these distinct characteristics of polynomial multiplications.

TABLE II
COEFFICIENT RANGES OR DISTRIBUTIONS OF POLYNOMIAL MULTIPLICATIONS IN ML-DSA.

PolyMul	Left Multiplicand	Right Multiplicand
$\mathbf{As}_1$	$\mathbb{Z}_q$	$[-\eta,\eta]$
$\mathbf{Ay},\mathbf{Az}$	$\mathbb{Z}_q$	$\mathbb{Z}_q$
$\boldsymbol{cs}_1,\boldsymbol{cs}_2$	B_τ	$[-\eta,\eta]$
$\boldsymbol{ct}_0$	B_τ	$(-2^{d-1},2^{d-1}]$
$\boldsymbol{ct}_1$	B_τ	$(-2^{\lceil\log_2 q\rceil-d},2^{\lceil\log_2 q\rceil-d}]$

B. Number-Theoretic Transform

NTT can be viewed as a Discrete Fourier Transform (DFT) defined over a finite field $\mathbb{Z}_q$, enabling accurate polynomial multiplication without loss of precision. The ring in ML-DSA is $R_q=\mathbb{Z}_q[X]/(X^n+1)$, and one must calculate the polynomial multiplication via the negative-wrapped convolution (NWC) in it [3]. For a polynomial $\boldsymbol{a}=\sum_{i=0}^{n-1}a_iX^i\in\mathbb{Z}_q[X]/(X^n+1)$, $\hat{\boldsymbol{a}}=\sum_{i=0}^{n-1}\hat{a}_iX^i$ denotes its representation in NTT domain. The definition of NTT and its inverse (INVNTT) based on the primitive $2n$-th root of unity, ψ, is detailed below:

$$\hat{a}_j=\sum_{i=0}^{n-1}\psi^{2ij+j}a_i \bmod q,$$

$$a_i=n^{-1}\sum_{j=0}^{n-1}\psi^{-(2ij+j)}\hat{a}_j \bmod q.$$

Given two polynomials $\boldsymbol{a}$ and $\boldsymbol{b}$, we can compute their product $\boldsymbol{c}=\boldsymbol{a}\cdot\boldsymbol{b}$ using negative-wrapped NTT. The product in the transformed domain can be calculated as

$$\boldsymbol{c}=\mathsf{INVNTT}((\mathsf{NTT}(\boldsymbol{a})\circ\mathsf{NTT}(\boldsymbol{b})),$$

where $\circ$ denotes element-wise multiplication.

The well-known Fast Fourier Transform (FFT) optimization reduces the complexity of DFT from $\mathcal{O}(n^2)$ to $\mathcal{O}(n\log n)$ using divide-and-conquer butterfly operations, originally proposed independently by Cooley-Tukey (CT) [38] and Gentleman-Sande (GS) [39]. Since $\mathbb{Z}_q$ contains a primitive $2n$-th root of unity ψ, the forward NTT can be computed as illustrated in Algorithm 4. Here, $brv_\mu(k)$ denotes the bit-reversal of the μ-bit integer k. For example, under the Chinese Remainder Theorem (CRT) map,

$$\boldsymbol{a} \bmod (x^{n/2}-\psi^{n/2})\mapsto$$
$$(\boldsymbol{a} \bmod (x^{n/4}-\psi^{n/4}),\ \boldsymbol{a} \bmod (x^{n/4}+\psi^{n/4})),$$

the CT butterfly transforms the pair $(a_i,a_{i+n/4})$ as

$$(a_i,a_{i+n/4})\leftarrow(a_i+\psi^{n/4}a_{i+n/4},a_i-\psi^{n/4}a_{i+n/4}),$$

which is typically used in the forward NTT with output in bit-reversed order. Conversely, the GS butterfly used in the inverse NTT is

$$(a_i,a_{i+n/4})\leftarrow\left(\frac{1}{2}(a_i+a_{i+n/4}),\frac{1}{2}\psi^{-n/4}(a_i-a_{i+n/4})\right).$$

For polynomial multiplication, after performing the CT butterfly forward NTT on both input polynomials, element-wise multiplication is carried out in the NTT domain. The inverse NTT with GS butterfly then transforms the product back to normal order.

Algorithm 4 Forward NTT

Require: Polynomial $\boldsymbol{a} = \sum_{i=0}^{n-1} a_i X^i \in R_q$, twiddle factors $\Psi[k] = \psi^{brv_\mu(k)} \bmod q$ for $0 < k < n, \mu = \log_2 n, \psi^{2n} \equiv 1 \bmod q$.
Ensure: Polynomial $\hat{\boldsymbol{a}} = \mathsf{NTT}(\boldsymbol{a})$.

$k \leftarrow 0$
$len \leftarrow n/2$
while $len > 0$ **do**
  for j from 0 to n by len **do**
    $\zeta \leftarrow \Psi[{+}{+}k]$
    for i from j to $j + len$ by 1 **do**
      $tmp \leftarrow \zeta \cdot a_{i+len}$
      $a_{i+len} \leftarrow a_i - tmp$
      $a_i \leftarrow a_i + tmp$
  $len \leftarrow len/2$
return $\hat{\boldsymbol{a}} = \sum_{i=0}^{n-1} a_i X^i$

C. Parallel Sparse Polynomial Multiplication (PSPM)

For polynomial multiplications involving the sparse polynomial $\boldsymbol{c}$, the multiplication can be efficiently implemented as simple additions or subtractions, which benefits greatly from processors with fast addition pipelines. Since $\boldsymbol{c}$ is multiplied with a polynomial vector, the non-zero coefficient positions of $\boldsymbol{c}$ affect the same indices across all vector elements. Additionally, the small coefficients of polynomials in $\mathbf{s}_1$ and $\mathbf{s}_2$ enable efficient parallel processing.

To further exploit parallelism, prior work [16], [26] proposed concatenating coefficients at the same index across multiple polynomials into a single machine word (e.g., `uint32_t` or `uint64_t`). These packed coefficients are converted to non-negative values for uniform vectorized operations. Based on whether the corresponding coefficient in $\boldsymbol{c}$ is $+1$ or -1, vectorized addition or subtraction is applied. Finally, the results are unpacked and stored back to the polynomial vector. The procedure is detailed in Algorithm 5. The parameters used in PSPM, derived from the parameter sets of ML-DSA and the characteristics of polynomial multiplication, are summarized in Table III.

TABLE III
PARAMETER CONFIGURATION FOR THE PSPM IMPLEMENTATION.

Scheme	Operation	τ	U	$2\tau U$	M	r
ML-DSA-44	$\boldsymbol{cs}_1$	39	2	156	2^8	4
	$\boldsymbol{cs}_2$	39	2	156	2^8	4
	$\boldsymbol{ct}_0$	39	2^{12}	319488	2^{19}	4
	$\boldsymbol{ct}_1$	39	2^{10}	79872	2^{17}	4
ML-DSA-65	$\boldsymbol{cs}_1$	49	4	392	2^9	5
	$\boldsymbol{cs}_2$	49	4	392	2^9	6
	$\boldsymbol{ct}_0$	49	2^{12}	401408	2^{19}	6
	$\boldsymbol{ct}_1$	49	2^{10}	100352	2^{17}	6
ML-DSA-87	$\boldsymbol{cs}_1$	60	2	240	2^8	7
	$\boldsymbol{cs}_2$	60	2	240	2^8	8
	$\boldsymbol{ct}_0$	60	2^{12}	491520	2^{19}	8
	$\boldsymbol{ct}_1$	60	2^{10}	122880	2^{17}	8

Algorithm 5 A parallel sparse polynomial multiplication (PSPM) algorithm with translations

Require: Sparse polynomial $\boldsymbol{c} = \sum_{i=0}^{n-1} c_i X^i \in B_\tau$; Polynomial vector with small-coefficient polynomial $\mathbf{a} = [\boldsymbol{a}^{(0)}, \ldots, \boldsymbol{a}^{(r-1)}]^T \in R_q^r$, where $\boldsymbol{a}^{(j)} = \sum_{i=0}^{n-1} a_i^{(j)} X^i \in R_q$.
Ensure: $\mathbf{u} = \boldsymbol{c} \cdot \mathbf{a} = [\boldsymbol{u}^{(0)}, \ldots, \boldsymbol{u}^{(r-1)}]^T \in R_q^r$, where $\boldsymbol{u}^{(j)} = \boldsymbol{c} \cdot \boldsymbol{a}^{(j)} = \sum_{i=0}^{n-1} u_i^{(j)} X^i \in R_q$

$\gamma := 2U \cdot \frac{M^r - 1}{M - 1}$ ▷ $U = \max(a_i), M = 2^{\lceil \log_2(1+2\tau U) \rceil}$
for i from 0 to $n-1$ **do**
  $w_i := 0$
  $v_i := 0$
  $v_{i-n} := 0$
  for j from 0 to $r-1$ **do**
    $v_i := v_i \cdot M + (U + a_i^{(j)})$
    $v_{i-n} := v_{i-n} \cdot M + (U - a_i^{(j)})$
for i from 0 to $n-1$ **do**
  if $c_i = 1$ **then**
    for j from 0 to $n-1$ **do**
      $w_j := w_j + v_{j-i}$
  if $c_i = -1$ **then**
    for j from 0 to $n-1$ **do**
      $w_j := w_j + (\gamma - v_{j-i})$
for i from 0 to $n-1$ **do**
  $t := w_i$
  for j from 0 to $r-1$ **do**
    $u_i^{(r-1-j)} := (t \bmod M) - \tau U \bmod q$
    $t := \lfloor t/M \rfloor$
return $\mathbf{u} = [\boldsymbol{u}^{(0)}, \ldots, \boldsymbol{u}^{(r-1)}]^T$

D. Discussion

Polynomial multiplication is a fundamental operation in lattice-based cryptography, and selecting an appropriate multiplication algorithm is crucial for balancing efficiency and implementation complexity. In this work, we mainly consider two representative approaches: NTT and PSPM. Table IV summarizes their key characteristics.

TABLE IV
COMPARISON OF NTT AND PSPM FOR POLYNOMIAL MULTIPLICATION

Aspect	NTT	PSPM
Complexity	$\mathcal{O}(n \log n)$	$\mathcal{O}(\tau n)$ (linear in sparsity)
Applicability	General polynomials in R_q	Sparse polynomials
Implementation	multiply, add/sub	and, shift, add/sub
Portability	High	Moderate

The NTT-based multiplication is standard for dense polynomial multiplication due to its asymptotic efficiency and well-established implementations. It leverages CT and GS techniques in finite fields, enabling large-degree polynomial multiplications with logarithmic complexity. Moreover, modern CPUs and hardware accelerators often provide specialized instructions or libraries optimizing NTT operations, further enhancing performance.

The PSPM method targets the special case where one polynomial is sparse. By replacing expensive multiplications

with additions and subtractions guided by the sparsity pattern, PSPM reduces arithmetic complexity and can leverage the CPU's fast addition pipelines. Additionally, packing multiple coefficients into machine words and applying vectorized operations increases parallelism and data throughput. However, the PSPM approach also brings several implementation challenges. The packing and unpacking of coefficients must be carefully managed to prevent overflow and ensure correctness, particularly since coefficients are converted into non-negative values for uniform vectorized processing. Additionally, aligning sparse coefficient indices across multiple polynomial vectors requires careful translation. These complexities complicate the design and hinder portability, as PSPM optimizations often depend on architecture-specific SIMD instructions. Consequently, targeted optimizations of PSPM on specific CPU architectures are both challenging and necessary.

IV. Optimized Implementation

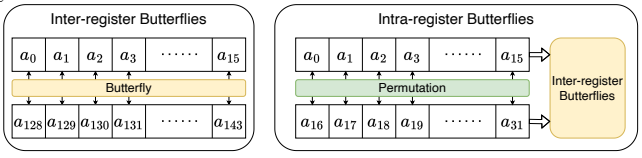
A. NTT Acceleration

The NTT constitutes a major computational bottleneck in ML-DSA, as polynomial multiplications involving the matrix $\mathbf{A}$ dominate the workload. To fully leverage the parallel execution capabilities of the ARMv9-A architecture, we implement a vectorized NTT using the SVE2 instruction set.

1) Register Blocking and Data Loading: In our design, $n = 256 = 16 \cdot 16$ coefficients of a polynomial are segmented into blocks that are mapped to 16 vector registers, each with a width of 512 bits (holding 16 32-bit coefficients). This register-blocking scheme allows an entire stage of butterfly operations to be computed with minimal load/store operations, thus reducing memory traffic and improving cache utilization. The vectorized NTT requires two distinct butterfly patterns. Figure 1 illustrates these two butterfly patterns during the vectorized NTT execution.

- *Inter-register butterflies*: These operations are performed between different vector registers. Each butterfly step begins with modular multiplication using the corresponding twiddle factors, followed by modular additions and subtractions. Inter-register operations allow multiple butterfly computations to proceed simultaneously, and the results are written in-place to avoid extra data movement.
- *Intra-register butterflies*: These operations are applied to elements within a single vector register. SVE2 permute instructions such as `uzp`, `zip` and `trn` are used to rearrange coefficients so that butterfly pairs occupy the same lane positions across two registers. Once aligned, computation instructions perform the butterfly computations in parallel, effectively enabling subsequent inter-register butterflies.

Fig. 1. Two butterfly patterns in NTT vectorization: inter-register and intra-register butterflies.



2) Modular Arithmetic: The NTT operates over $\mathbb{Z}_q$, requiring modular reduction after multiplication and addition operations to ensure final results remain within the range of $[0, q)$ and avoid intermediate data overflow.

When one operand in the multiplication is known in advance (e.g., the twiddle factors), we adopt Barrett multiplication [25] to perform modular multiplication efficiently. Algorithm 6 presents our SVE2-optimized Barrett multiplication for this case. When both operands are unknown, Montgomery multiplication is employed to avoid costly integer division. Algorithm 7 shows our SVE2-optimized Montgomery multiplication, which operates entirely in the vector domain.

The use of modular multiplication also ensures that additions and subtractions in NTT do not overflow the 32-bit lane, eliminating the need for extra reductions in these operations. Since the final step of the inverse NTT is a modular multiplication, the result of an NTT-based polynomial multiplication is guaranteed to lie within $\mathbb{Z}_q$.

Algorithm 6 Barrett multiplication with one known operand and one unknown operand

Require: Coefficient register `a` $= (a_0, \ldots, a_{15}) \in \mathbb{Z}_q$, constant operand `b` $= (b_0, \ldots, b_{15}) \in \mathbb{Z}_q$, precomputed `bR` $= (b'_0, \ldots, b'_{15})$ with $b'_i = \lfloor \frac{b_i \cdot 2^{32}}{2q} \rceil$, modulus register `q`, temporary register `t`, predicate register `p0`.
Ensure: Product register `c` $= (a_0 b_0, \ldots, a_{15} b_{15}) \in \mathbb{Z}_q$

1: `mul c.s, a.s, b.s` ▷ $z_i := a_i b_i \bmod \beta$
2: `sqrdmulh t.s, a.s, bR.s` ▷ $t_i := \lfloor \frac{2a_i \cdot b'_i}{2^{32}} \rfloor$
3: `mls c.s, p0/m,t.s, q.s` ▷ $c_i = t_i - q t_i$

Algorithm 7 Montgomery multiplication with two unknown operands

Require: Coefficient register `a` $\in \mathbb{Z}_q$, `b` $\in \mathbb{Z}_q$ with each element 32-bit, parameters register `q`, `qinv` $= (q^{-1} \bmod \beta, \ldots, q^{-1} \bmod \beta)$ with each element 32-bit, where $\beta = 2^{32}$, temporary register `tmp`, `tmp2`, `t`.
Ensure: Product register `c` $= (a_0 b_0 \beta^{-1}, \ldots, a_{15} b_{15} \beta^{-1}) \in \mathbb{Z}_q$, where $\beta^{-1} = 2^{-32} \mod q$.

1: `smullb tmp.d, a.s, b.s`
2: `smullt tmp2.d, a.s, b.s`
3: `trn1 t.s, tmp.s, tmp2.s` ▷ $t_i := a_i b_i \bmod \beta$
4: `mul t.s, t.s, qinv.s` ▷ $t_i := t_i q^{-1} \bmod \beta$
5: `smlslb tmp.d, t.s, q.s`
6: `smlslt tmp2.d, t.s, q.s`
7: `trn2 c.s, tmp.s, tmp2.s` ▷ $c_i := \lfloor \frac{a_i b_i - t_i q}{\beta} \rfloor$

3) Memory Access Pattern and Prefetching: In our design, we retain polynomial coefficients in vector registers across multiple consecutive butterfly stages whenever register capacity allows. This approach eliminates redundant load and store operations, thereby reducing both instruction count and memory traffic. By maximizing register reuse, we exploit temporal locality, ensuring that recently computed values remain available for immediate reuse without incurring memory access latency. This not only improves computational throughput but also alleviates pressure on cache and memory bandwidth. For

the twiddle factors, we reorganize their layout in memory to precisely match the sequential access pattern of each CT and GS butterfly. In this scheme, the data for an entire layer can be fetched using contiguous vector load instructions, avoiding the need for stride or irregular memory accesses. Such contiguous access improves cache-line utilization, minimizes load latency, and ensures better alignment for vectorization.

B. PSPM Optimization

The PSPM method is tailored for the special case where one multiplicand is sparse such as $\boldsymbol{c}$ in ML-DSA's signing procedure. Instead of performing general-purpose $\mathcal{O}(n \log n)$ NTT-based multiplication, PSPM reduces the operation to a sequence of additions and subtractions guided by the sparsity pattern, thus significantly lowering arithmetic complexity.

The vectorized PSPM algorithm leverages SVE2 to accelerate the computation of $\boldsymbol{c}\mathbf{s}_1 + \mathbf{y}$, $\mathbf{w} - \boldsymbol{c}\mathbf{s}_2$, and $\boldsymbol{c}\mathbf{t}_0$, while incorporating an early rejection mechanism. Algorithm 8 illustrates the procedure using $\boldsymbol{c}\mathbf{s}_1 + \mathbf{y}$ as an example. The polynomial vector $\mathbf{s}_1$ is packed into a $2n$-element array v, which enables SIMD operations across multiple coefficients. During accumulation, each nonzero coefficient of the sparse polynomial $\boldsymbol{c}$ is examined: if $c_i = 1$, v is added; if $c_i = -1$, v is subtracted. In this way, the partial products are accumulated directly in vector registers, yielding the packed representation of $\boldsymbol{c}\mathbf{s}_1$. After accumulation step, the intermediate result $\boldsymbol{z}^{(j)}$ is checked against the norm bound using SVE2 predicate registers. If any lane fails the comparison, the algorithm immediately terminates with an invalid output, forcing the signature generation process to restart from the rejection loop. This early rejection strategy avoids unnecessary computations, while vectorization with SVE2 significantly reduces branch mispredictions and improves throughput compared to the scalar baseline.

1) Predicate-based early rejection: Our vectorized PSPM leverages predicate registers to implement early rejection [16] efficiently. Instead of completing the entire accumulation, the algorithm evaluates whether the intermediate result violates the norm bound at regular intervals. Based on empirical rejection rates of $\boldsymbol{c}\mathbf{s}_1 + \mathbf{y}$ and $\mathbf{w} - \boldsymbol{c}\mathbf{s}_2$, we compute the one with the higher expected rejection probability first, so that the signing process can restart earlier on average. Furthermore, instead of checking after every coefficient, we evaluate the rejection condition every $n/4$ coefficients, which strikes a balance between minimizing overhead and enabling early termination. The use of predicate registers avoids scalar if/else branches, thereby reducing branch and improving overall efficiency. The execution flow before and after introducing the *early check* mechanism is illustrated in Figure 2.

2) Chunk-based accumulation: Due to the limited number of vector registers, it is not feasible to keep all n coefficients in registers during the accumulation phase of polynomial multiplication. To address this limitation, we adopt a chunk-based accumulation strategy. Specifically, the coefficients are divided into fixed-size chunks (e.g., $n/2$ or $n/4$), and each chunk is processed independently. For example, when processing $\mathbf{s}_1$ and $\mathbf{s}_2$, the packed coefficients reside in .s lanes, with 16 coefficients per register. In Algorithm 8 (lines 5, 7-8), one loop processes $n/2$ coefficients. Considering temporary registers, each chunk uses 8 registers for accumulation results, 8 registers for packed coefficients, 8 temporary registers, and 1 constant register, totaling 25 registers. Without the chunk-based strategy, the required registers would double, exceeding the architectural limit. Similarly, when processing $\mathbf{t}_0$, the packed coefficients are 64-bit with 8 coefficients per register, so each chunk processes $n/4$ coefficients to respect the register constraints. After computing the partial accumulation for one chunk, each polynomial coefficient of the vector is evaluated and checked for validity (Algorithm 8, lines 9-16). The next chunk is then loaded, accumulated, and evaluated until all coefficients are processed.

This approach ensures that the register footprint remains within hardware limits, avoiding costly register spills introduced by the compiler. At the same time, chunking naturally allows intermediate rejection checks between segments, further reducing wasted computations during signing. Overall, chunk-based accumulation balances hardware constraints and algorithmic efficiency, making it well-suited for vectorized implementations.

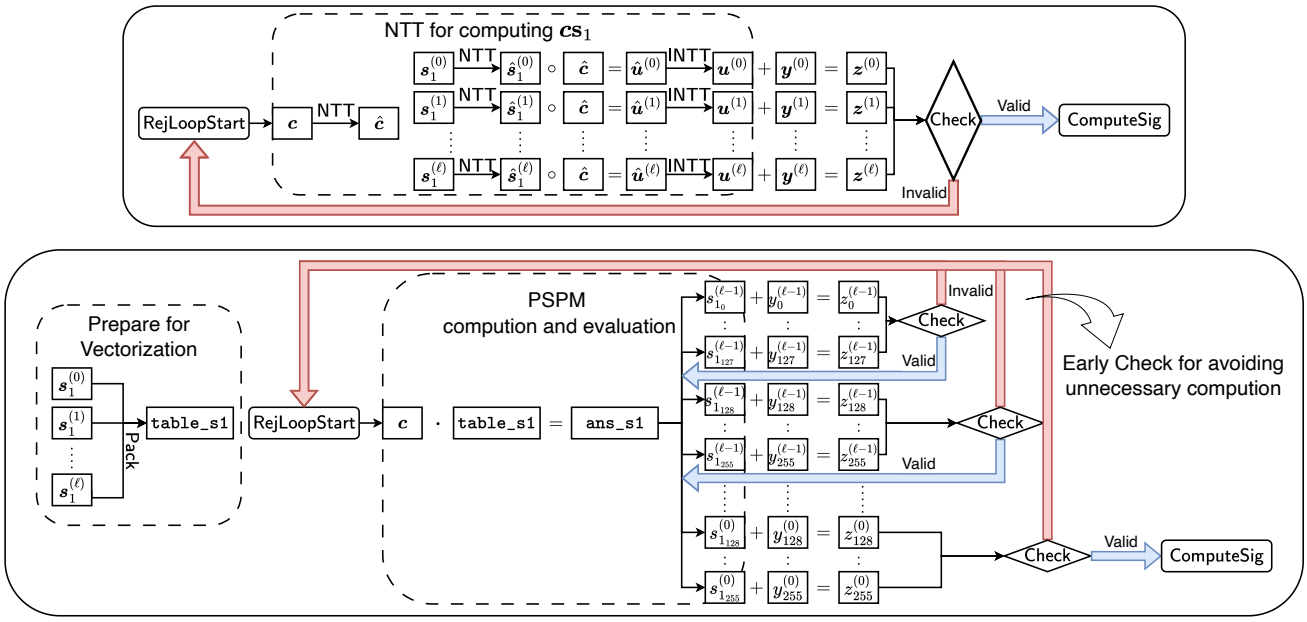
3) Architecture-Specific Considerations: While PSPM benefits significantly from vectorization, its performance is sensitive to the choice of vector width and instruction set. For instance, SVE2's lane-wise add/sub operations on 32-bit integers map naturally to data layout for PSPM, but equivalent implementations on other SIMD architectures (e.g., NEON or AVX2) require different packing strategies due to smaller vector widths. Moreover, handling coefficient packing/unpacking

Algorithm 8 Vectorized PSPM with SVE2 for computing $\boldsymbol{c}\mathbf{s}_1 + \mathbf{y}$ with early norm check

Require: Sparse polynomial $\boldsymbol{c} = \sum_{i=0}^{n-1} c_i X^i \in B_\tau$
Require: Packed array $v[2n]$ representing the polynomial vector $\mathbf{s}_1$
Require: Polynomial vector $\mathbf{y} = [\boldsymbol{y}^{(0)}, \ldots, \boldsymbol{y}^{(\ell-1)}]^T$, where $\boldsymbol{y}^{(j)} = \sum_{i=0}^{n-1} y_i^{(j)} X^i \in R_q$
Ensure: $\mathbf{z} = \boldsymbol{c}\mathbf{s}_1 + \mathbf{y}$ if $\|\mathbf{z}\| < \gamma_1 - \beta$, otherwise *invalid*

1: $p_0 \leftarrow$ `svptrue()` ▷ initialize predicate for active lanes
2: `w.s` $\leftarrow 0$, `gamma.s` $\leftarrow \gamma$ ▷ SVE Vectors
3: **for** $i \leftarrow 0$ **to** $n-1$ **do**
4: **if** $c_i = 1$ **then**
5: `add w.s, w.s, v.s` ▷ chunk-based acc.
6: **else if** $c_i = -1$ **then**
7: `sub tmp.s, gamma.s, v.s`
8: `add w.s, w.s, tmp.s` ▷ chunk-based acc.
9: **for** $j \leftarrow \ell - 1$ **downto** 0 **do**
10: $\boldsymbol{t} \leftarrow (\mathrm{w} \bmod M) - \tau U$ ▷ `svand` and `svsub`
11: $\boldsymbol{z}^{(j)} \leftarrow \boldsymbol{t} + \boldsymbol{y}^{(j)}$ ▷ `svadd`
12: $p_1 \leftarrow (\boldsymbol{z}^{(j)} < \gamma_1 - \beta)$ ▷ `svcmplt`, predicate p_0
13: $p_2 \leftarrow (\boldsymbol{z}^{(j)} > \beta - \gamma_1)$ ▷ `svcmpgt`, predicate p_1
14: **if** not all lanes active in p_2 **then**
15: **return** *invalid*
16: `lsr w.s, w.s, #` $(\log_2 M)$
17: **return** $\mathbf{z} = [\boldsymbol{z}^{(0)}, \ldots, \boldsymbol{z}^{(\ell-1)}]^T$

Fig. 2. Execution flow of $\boldsymbol{cs}_1$ computation in the rejection loop before and after applying *early check* mechanism.



without overflow demands careful modular arithmetic design, especially when coefficients are mapped to non-negative representations for uniform processing. SVE2 predicate registers provide lane-wise control without relying on scalar branching, which is particularly beneficial for early rejection, where conditional updates are frequently required.

4) Complexity and parallelism analysis: The scalar PSPM has complexity $O(\tau n)$, where n is the polynomial degree. In the vectorized version, the arithmetic is executed in parallel across `VL` lanes, where `VL` is the SVE2 vector length. Thus, the effective complexity is reduced to $O(\tau n/\mathtt{VL})$. Since `VL` is hardware-dependent, the performance scales with the underlying microarchitecture. In practice, the performance gain also depends on the rejection probability: when early rejection is frequent, the expected computation cost is further reduced. This parallelized and rejection-aware design makes the vectorized PSPM highly efficient.

C. Optimized Signing Procedure

Building upon the vectorized polynomial multiplication techniques introduced in the previous sections, we integrate these optimizations into the signing procedure to improve overall performance. Specifically, the NTT-based multiplication can be directly employed as a drop-in replacement for the standard polynomial multiplication module in the signing loop. In contrast, the PSPM-based approach requires a more adaptive modification of the signing procedure. As illustrated in Fig. 2, the conventional NTT-based method completes all computations before performing the validity check, which may result in wasted work if the generated signature is rejected at an early stage. By adopting PSPM with early check, the signing loop is divided into multiple chunks, and a partial norm check is performed after processing each chunk. If the check fails, the computation is immediately discarded, avoiding unnecessary operations for the remaining coefficients.

The chunk-based early rejection scheme not only reduces redundant computations but also creates new opportunities for pipeline optimization. In particular, by decoupling computation from validation, the signing procedure can exploit instruction-level parallelism more effectively. For example, while the computation of chunk $i+1$ is underway, the validity check for chunk i can be executed in parallel.

From a micro-architectural perspective, such a design allows better utilization of the wide execution units in ARMv9-A processors. Predicate registers can be used to trigger the rejection test without branching, while arithmetic operations for subsequent chunks continue in parallel. This dual-path execution not only hides the latency of the norm check but also balances the computational load across the pipeline stages.

V. Performance Evaluation

A. Benchmarking Setup

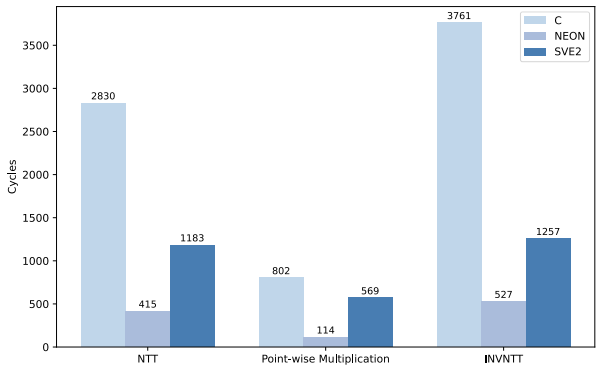
All benchmarks were conducted on a MacBook Pro equipped with Apple M4 Pro SoC. The code was compiled with `Apple clang 16.0` using the flags `-march=armv9.2-a+sme2` and `-O3` to enable SVE2 instructions and apply high-level optimizations. All SVE2 assembly routines were executed in streaming SVE mode, controlled via `smstart` and `smstop`. We used the reference implementation [4] and the SOTA implementation [25] as the baseline and compared them against our optimized version. Reported performance numbers correspond to the median cycle counts over 10,000 runs, measured in clock cycles.

B. Performance of Polynomial Multiplication

1) NTT-based polynomial multiplicaiton: Figure 3 presents the performance comparison of NTT-based polynomial multiplication, including NTT, point-wise multiplication, and inverse NTT across three implementations. The reference

implementation from [4] incurs the highest cycle counts. The NEON-optimized implementation [25] significantly reduces this cost, achieving a speedup of over $6.8\times$ in NTT. Our SVE2-based implementation also accelerates computation compared to the reference. Specifically, the NTT routine is $2.4\times$ faster than the reference, though approximately $2.9\times$ slower than the NEON version. For INVNTT, our design achieves a $3.0\times$ improvement over the reference, yet still falls short of the NEON implementation. These results demonstrate the effectiveness of SVE2 vectorization, even though NEON maintains an advantage for extremely latency-sensitive, short-vector operations. Moreover, our approach provides robust speedups over the reference implementation and offers a unified optimization framework applicable to NTT-based polynomial accumulation. The observed performance gap may also be partially attributed to the relatively recent SVE2 instruction set, whose pipeline adaptation on the target device is not yet fully optimized. Importantly, our NTT method is general-purpose and readily portable to future SVE2-enabled platforms.

Fig. 3. Cycle counts of NTT-based polynomial multiplication.



2) Vectorized PSPM polynomial multiplicaiton: This subsection discusses the comparison of vectorized PSPM and NTT-based polynomial multiplication across different ML-DSA parameter sets. For the NTT-based comparison, we select the implementation [25] that demonstrates the best performance in each parameter set as the baseline. The detailed results are presented in Fig. 4 and Fig. 5. It can be observed that PSPM yields substantial performance improvements over the NTT baseline, especially for the $\mathbf{cs}1 + \mathbf{cs}2$ module, which dominates the computational cost in the signing procedure. For the $\boldsymbol{cs}_1 + \boldsymbol{cs}_2$ modules, our SVE2-based implementation achieves substantial reductions in cycle counts compared to the NEON-optimized work, with optimization rates of 81.9%, 70.5%, and 58.6% for ML-DSA-44, ML-DSA-65, and ML-DSA-87, respectively. This indicates that the PSPM optimization is particularly effective for smaller parameter sets, while the relative gain decreases for larger sets due to increased memory access and accumulation overhead. For the $\boldsymbol{ct}_0$ module, the speedup is also significant for ML-DSA-44, achieving a 76.8% reduction, but drops notably to 26.0% and 24.1% for ML-DSA-65 and ML-DSA-87. This trend suggests that, for modules involving higher-dimensional polynomial vectors and larger numbers of ± 1 entries in $\mathbf{c}$, the performance gains of PSPM are somewhat limited. Nevertheless, the results demonstrate that PSPM-based vectorization provides consistent performance improvements across all parameter sets, and establishes a foundation for SVE2-enabled polynomial multiplication.

Fig. 4. Comparison of $\mathbf{cs}_1 + \mathbf{cs}_2$ across different parameter sets

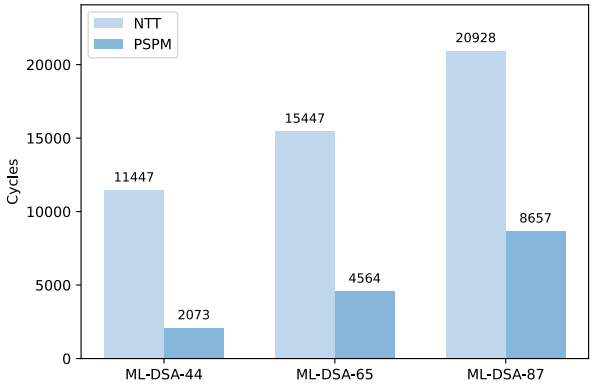
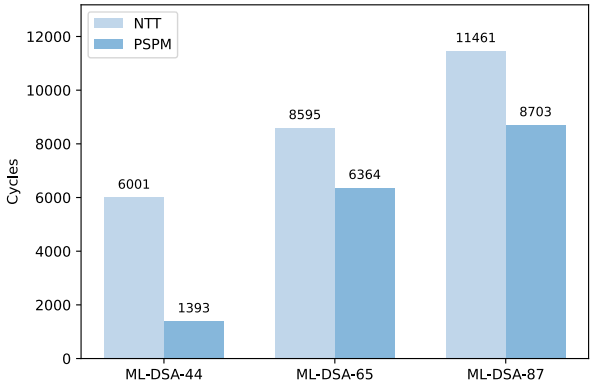


Fig. 5. Comparison of $\mathbf{ct}_0$ across different parameter sets.



C. Overall Performance

Table V reports the cycle counts of ML-DSA signature generation across all parameter sets. Compared to the reference implementation [3], the NEON-optimized version [25] already achieves significant acceleration. Our SVE2-based implementation further improves performance, reaching cycle reductions of 70-72% relative to the reference. The results indicate that SVE2 vectorization provides consistent speedup across all security levels, outperforming the NEON-optimized version, and demonstrates the effectiveness of our unified optimization strategy for signature generation.

VI. Conclusion

In this work, we have presented two vectorized polynomial multiplication schemes on the ARMv9-A architecture by leveraging the SVE2 instruction set. Specifically, we introduced an architecture-aware NTT implementation that exploits wide vector registers and modular arithmetic optimizations, and we proposed a PSPM-based method tailored for efficient vectorization. To further improve the signing process, we incorporated an early check mechanism into the PSPM

TABLE V
PERFORMANCE OF SIGNATURE GENERATION OF ML-DSA ACROSS ALL PARAMETER SETS ON ARMV9-A.

Implementation	ML-DSA-44	Speed up vs. [4]	ML-DSA-65	Speed up vs. [4]	ML-DSA-87	Speed up vs. [4]
Reference [4]	678503		1106225		1376613	
Becker et al. [25]	213168	68.6%	327874	70.4%	397530	71.1%
Ours	198932	70.7%	316752	71.4%	385457	72.0%

scheme, enabling chunk-based rejection sampling that avoids unnecessary computations and shortens the rejection loop.

Our experimental results on real ARMv9-A hardware demonstrate that these optimizations significantly accelerate polynomial arithmetic, which translates into substantial improvements in ML-DSA signing performance. Beyond raw speedups, the proposed early check design also highlights the potential of pipeline-level parallelism and instruction-level overlap between computation and validation. These findings confirm the practicality of deploying post-quantum cryptography on next-generation ARM processors and provide a foundation for future PQC engineering that fully exploits advanced architectural features such as SVE2.

REFERENCES

[1] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM J. Comput.*, vol. 26, no. 5, pp. 1484–1509, 1997.

[2] L. K. Grover, "A fast quantum mechanical algorithm for database search," in *Proceedings of the Twenty-Eighth Annual ACM Symposium on the Theory of Computing, Philadelphia, Pennsylvania, USA, May 22-24, 1996*, G. L. Miller, Ed. ACM, 1996, pp. 212–219. [Online]. Available: https://doi.org/10.1145/237814.237866

[3] S. Bai, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, P. Schwabe, G. Seiler, and D. Stehl, "Crystals-dilithium: Algorithm specifications and supporting documentation (version 3.1)," *Submission to the NISTs post-quantum cryptography standardization process*, 2021. [Online]. Available: https://pq-crystals.org/dilithium/data/dilithium-specification-round3-20210208.pdf

[4] NIST, "Fips 204: Module-lattice-based digital signature standard," https://csrc.nist.gov/pubs/fips/204/final, 2024.

[5] V. Lyubashevsky, "Fiat-shamir with aborts: Applications to lattice and factoring-based signatures," in *Advances in Cryptology - ASIACRYPT 2009, 15th International Conference on the Theory and Application of Cryptology and Information Security, Tokyo, Japan, December 6-10, 2009. Proceedings*, ser. Lecture Notes in Computer Science, M. Matsui, Ed., vol. 5912. Springer, 2009, pp. 598–616.

[6] ——, "Lattice signatures without trapdoors," in *Advances in Cryptology - EUROCRYPT 2012 - 31st Annual International Conference on the Theory and Applications of Cryptographic Techniques, Cambridge, UK, April 15-19, 2012. Proceedings*, ser. Lecture Notes in Computer Science, D. Pointcheval and T. Johansson, Eds., vol. 7237. Springer, 2012, pp. 738–755.

[7] A. Langlois and D. Stehlé, "Worst-case to average-case reductions for module lattices," *Des. Codes Cryptogr.*, vol. 75, no. 3, pp. 565–599, 2015. [Online]. Available: https://doi.org/10.1007/s10623-014-9938-4

[8] M. Ajtai, "Generating hard instances of lattice problems (extended abstract)," in *Proceedings of the Twenty-Eighth Annual ACM Symposium on the Theory of Computing, Philadelphia, Pennsylvania, USA, May 22-24, 1996*, G. L. Miller, Ed. ACM, 1996, pp. 99–108. [Online]. Available: https://doi.org/10.1145/237814.237838

[9] Z. Wu, R. Chen, Y. Wang, Q. Wang, and W. Peng, "An efficient hardware implementation of crystal-dilithium on FPGA," in *Information Security and Privacy - 29th Australasian Conference, ACISP 2024, Sydney, NSW, Australia, July 15-17, 2024, Proceedings, Part II*, ser. Lecture Notes in Computer Science, T. Zhu and Y. Li, Eds., vol. 14896. Springer, 2024, pp. 64–83.

[10] T. Wang, C. Zhang, X. Zhang, D. Gu, and P. Cao, "Optimized hardware-software co-design for kyber and dilithium on RISC-V soc FPGA," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2024, no. 3, pp. 99–135, 2024.

[11] S. Mandal and D. B. Roy, "Kid: A hardware design framework targeting unified NTT multiplication for crystals-kyber and crystals-dilithium on FPGA," in *37th International Conference on VLSI Design and 23rd International Conference on Embedded Systems, VLSID 2024, Kolkata, India, January 6-10, 2024*. IEEE, 2024, pp. 455–460.

[12] Y. Ouyang, Y. Zhu, W. Zhu, B. Yang, Z. Zhang, H. Wang, Q. Tao, M. Zhu, S. Wei, and L. Liu, "Falconsign: An efficient and high-throughput hardware architecture for falcon signature generation," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2025, no. 1, pp. 203–226, 2025. [Online]. Available: https://doi.org/10.46586/tches.v2025.i1.203-226

[13] Z. Ye, J. Huang, T. Huang, Y. Bai, J. Li, H. Zhang, G. Li, D. Chen, R. C. C. Cheung, and K. Huang, "PQNTRU: acceleration of ntru-based schemes via customized post-quantum processor," *IEEE Trans. Computers*, vol. 74, no. 5, pp. 1649–1662, 2025. [Online]. Available: https://doi.org/10.1109/TC.2025.3540647

[14] R. Xu, D. He, M. Luo, C. Peng, and X. Zeng, "Optimizing dilithium implementation with AVX2/-512," *ACM Trans. Embed. Comput. Syst.*, vol. 23, no. 6, pp. 98:1–98:30, 2024.

[15] J. Zheng, H. Zhu, Z. Song, Z. Wang, and Y. Zhao, " Optimized Vectorization Implementation of CRYSTALS-Dilithium ," *IEEE Transactions on Dependable and Secure Computing*, no. 01, pp. 1–13, May 2025. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/TDSC.2025.3575915

[16] S. Shen, H. Yang, W. Li, and Y. Zhao, "cuml-dsa: Optimized signing procedure and server-oriented GPU design for ML-DSA," *IEEE Trans. Dependable Secur. Comput.*, vol. 22, no. 3, pp. 2295–2307, 2025. [Online]. Available: https://doi.org/10.1109/TDSC.2024.3494835

[17] Z. Wang, X. Dong, Y. Kang, H. Chen, and Q. Wang, "An example of parallel merkle tree traversal: Post-quantum leighton-micali signature on the gpu," *ACM Trans. Archit. Code Optim.*, vol. 21, no. 3, Sep. 2024. [Online]. Available: https://doi.org/10.1145/3659209

[18] W. Lee, R. K. Zhao, R. Steinfeld, A. Sakzad, and S. O. Hwang, "High throughput lattice-based signatures on gpus: Comparing falcon and mitaka," *IEEE Trans. Parallel Distributed Syst.*, vol. 35, no. 4, pp. 675–692, 2024. [Online]. Available: https://doi.org/10.1109/TPDS.2024.3367319

[19] Z. Wang, X. Dong, H. Chen, Y. Kang, and Q. Wang, "Cuspx: Efficient gpu implementations of post-quantum signature sphincs¡sup¿+¡/sup¿," *IEEE Trans. Comput.*, vol. 74, no. 1, p. 1528, Jan. 2025. [Online]. Available: https://doi.org/10.1109/TC.2024.3457736

[20] D. O. C. Greconici, M. J. Kannwischer, and A. Sprenkels, "Compact dilithium implementations on cortex-m3 and cortex-m4," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2021, no. 1, pp. 1–24, 2021.

[21] A. Abdulrahman, V. Hwang, M. J. Kannwischer, and A. Sprenkels, "Faster kyber and dilithium on the cortex-m4," in *Applied Cryptography and Network Security - 20th International Conference, ACNS 2022, Rome, Italy, June 20-23, 2022, Proceedings*, ser. Lecture Notes in Computer Science, G. Ateniese and D. Venturi, Eds., vol. 13269. Springer, 2022, pp. 853–871. [Online]. Available: https://doi.org/10.1007/978-3-031-09234-3_42

[22] J. Huang, A. Adomnicai, J. Zhang, W. Dai, Y. Liu, R. C. C. Cheung, Ç. K. Koç, and D. Chen, "Revisiting keccak and dilithium implementations on armv7-m," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2024, no. 2, pp. 1–24, 2024. [Online]. Available: https://doi.org/10.46586/tches.v2024.i2.1-24

[23] M. J. Kannwischer, M. Krausz, R. Petri, and S.-Y. Yang, "pqm4: Benchmarking NIST additional post-quantum signature schemes on microcontrollers," Cryptology ePrint Archive, Paper 2024/112, 2024. [Online]. Available: https://eprint.iacr.org/2024/112

[24] T. Pornin, "Improved (again) key pair generation for falcon, BAT and hawk," *IACR Cryptol. ePrint Arch.*, p. 1239, 2025. [Online]. Available: https://eprint.iacr.org/2025/1239

[25] H. Becker, V. Hwang, M. J. Kannwischer, B. Yang, and S. Yang, "Neon NTT: faster dilithium, kyber, and saber on cortex-a72 and apple M1," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2022, no. 1, pp. 221–244, 2022.

[26] J. Zheng, F. He, S. Shen, C. Xue, and Y. Zhao, "Parallel small polynomial multiplication for dilithium: A faster design and implementation," in *Annual Computer Security Applications Conference, ACSAC 2022, Austin, TX, USA, December 5-9, 2022*. ACM, 2022, pp. 304–317.
[27] D. T. Nguyen and K. Gaj, "Fast falcon signature generation and verification using armv8 NEON instructions," in *Progress in Cryptology - AFRICACRYPT 2023 - 14th International Conference on Cryptology in Africa, Sousse, Tunisia, July 19-21, 2023, Proceedings*, ser. Lecture Notes in Computer Science, N. E. Mrabet, L. D. Feo, and S. Duquesne, Eds., vol. 14064. Springer, 2023, pp. 417–441. [Online]. Available: https://doi.org/10.1007/978-3-031-37679-5_18
[28] Arm, "Arm architecture reference manual for a-profile architecture," https://developer.arm.com/documentation/ddi0487/latest, 2024.
[29] ——, "Learn the architecture - migrate neon to sve," https://developer.arm.com/documentation/102131/0100/?lang=en, 2022.
[30] ——, "Learn the architecture - introducing sve2 guide," https://developer.arm.com/documentation/102340/0100/?lang=en, 2024.
[31] Apple, "Apple introduces m4 chip," https://www.apple.com/newsroom/2024/05/apple-introduces-m4-chip/, 2024.
[32] MediaTek, "vivo v27 powered by mediatek dimensity 7200," https://www.mediatek.com/tek-talk-blogs/vivo-v27-powered-by-mediatek-dimensity-7200, March 28, 2023.
[33] SAMSUNG, "Exynos 1580," https://semiconductor.samsung.cn/processor/mobile-processor/exynos-1580/, 2024.
[34] Arm, "Arm drives next-generation performance for iot with worlds first armv9 edge ai platform," https://newsroom.arm.com/news/armv9-edge-ai-platform, February 26, 2025.
[35] H. Becker and M. J. Kannwischer, "Hybrid scalar/vector implementations of keccak and sphincs+ on aarch64," in *Progress in Cryptology INDOCRYPT 2022: 23rd International Conference on Cryptology in India, Kolkata, India, December 1114, 2022, Proceedings*. Berlin, Heidelberg: Springer-Verlag, 2022, p. 272293. [Online]. Available: https://doi.org/10.1007/978-3-031-22912-1_12
[36] T. Güneysu, V. Lyubashevsky, and T. Pöppelmann, "Practical lattice-based cryptography: A signature scheme for embedded systems," in *Cryptographic Hardware and Embedded Systems - CHES 2012 - 14th International Workshop, Leuven, Belgium, September 9-12, 2012. Proceedings*, ser. Lecture Notes in Computer Science, E. Prouff and P. Schaumont, Eds., vol. 7428. Springer, 2012, pp. 530–547.
[37] S. Bai and S. D. Galbraith, "An improved compression technique for signatures based on learning with errors," in *Topics in Cryptology - CT-RSA 2014 - The Cryptographer's Track at the RSA Conference 2014, San Francisco, CA, USA, February 25-28, 2014. Proceedings*, ser. Lecture Notes in Computer Science, J. Benaloh, Ed., vol. 8366. Springer, 2014, pp. 28–47.
[38] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex fourier series," *Mathematics of computation*, vol. 19, no. 90, pp. 297–301, 1965.
[39] W. M. Gentleman and G. Sande, "Fast fourier transforms: For fun and profit," in *Proceedings of the November 7-10, 1966, Fall Joint Computer Conference*, ser. AFIPS '66 (Fall). New York, NY, USA: Association for Computing Machinery, 1966, p. 563578.